BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 229 · Programming paradigms

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is an object?

Q2. What does a constructor do?

Q3. What does a return statement do?

Q4. What property distinguishes a pure function from a method that changes a shared counter?

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check my understanding

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

paradigm — Overall style for organising programs.

procedural — Organises a program around sequences of operations and subroutines.

object-oriented — Encapsulates state and behaviour into objects.

functional — Emphasises functions applied to values and reduced side effects.

declarative — Expresses desired results rather than imperative step sequences.

New learning

Procedural — Instructions and procedures operate on arguments or shared data. A function can be reused independently.

Object-oriented — The Account object combines state (Balance) and behaviour (Deposit). With inheritance, a subclass can reuse and specialise a class. A virtual method overridden in a derived class supports polymorphism: the actual object decides which implementation runs.

Functional style — Values are transformed using expressions/functions, often emphasising avoiding mutation of existing input data.

Worked example

Classify a function Square(x) that returns x*x without storing object state.

1. Predict — It is procedural when used as a standalone function in a procedural programme.

2. Model response — Square(3) returns 9; it does not change object state. C# can use more than one paradigm.

3. Follow the declared and actual types — Both variables are declared as BaseMessage . Their actual objects are EmailMessage and TextMessage , each inheriting from the base class.

4. Why is virtual important? — The base method is virtual and both derived classes override it. The language dispatches to the actual object's implementation.

5. Output and design consequence — Outputs Email then Text . The caller can add another derived message class without writing separate type-checking branches.

We Do · Practise with guidance

Classify a Score object storing Points and updating it using Add(2).

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

```
var score=new Score(5);
score.Add(2);
// score.Points is 7
```

Predict

The Score object holds Points alongside a method that changes its state.

Model response

This is object-oriented design; after Add(2), its Points becomes 7.

Does a new method behave like an override?

```
using System;
BaseReport report = new ShortReport();
Console.WriteLine(report.Title());

class BaseReport {
  public string Title() => "Base";
}
class ShortReport : BaseReport {
  public new string Title() => "Short";
}
```

What prints?

Track the declared type of report and the keyword used on Title() .

There is no virtual dispatch

new hides the base method; it does not override it. Calls through a BaseReport reference bind to the base method here.

Output and correction

The output is Base . Make BaseReport.Title() virtual , and mark ShortReport.Title() override ; then the same base-typed call outputs Short .

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Classify a program using standalone functions with arguments and returns. Give one advantage.

Question 2

Compare procedural and object-oriented account implementations, then explain how an overridden method on a derived SavingsAccount can be called through an Account reference.

Question 3

Explain why `values.Select(x=>x*2)` can be considered functional-style programming, and one limitation of labelling C# purely functional.

Exit Ticket · Answer before checking

A language such as C# can support both procedural and object-oriented programming.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. A particular instance of a class.

Q2. Initialises a new object.

Q3. Ends the function and passes a result to its caller.

Q4. A pure function has no externally observable side effects and returns the same result for the same inputs.

You Do pages · Independent questions

Q1. [4] Procedural [1]; operations organised in subroutines [1]; parameters carry inputs [1]; functions can be reused/tested independently [1].

Q2. [6] Procedural code passes state to functions [2]; OOP stores account state with methods [2]; when a base method is virtual and the subclass overrides it, an Account reference to a SavingsAccount uses the SavingsAccount implementation [2].

Q3. [6] Applies a function to each element [2], returns transformed values [1], can avoid mutating original list [1]; C# also supports procedural code and mutable objects [1], so language/paradigm are not identical [1].

Exit Ticket · Model answer

True. Programming paradigms overlap within multi-paradigm languages.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: